

5-33

(1)

$$RTT_s = 1.5s$$

由于是第一次测量，所以 $RTT_D = 1.5s/2 = 0.75s$

$$RTO = RTT_s + 4RTT_D = 1.5s + 4 \times 0.75s = 4.5s$$

(2)

$$RTT_s = (1 - \alpha)(\text{旧的 } RTT_s) + \alpha(\text{新的 RTT 样本})$$

$$\begin{aligned} &= (1 - 0.125) \times 1.5s + 0.125 \times 2.5s \\ &= 1.625s \end{aligned}$$

$$\text{新的 } RTT_D = (1 - \beta)(\text{旧的 } RTT_D) + \beta |RTT_s - \text{新的 RTT 样本}|$$

$$\begin{aligned} &= (1 - 1/4) \times 0.75s + 1/4 \times |1.625 - 2.5|s \\ &= 0.78125s \end{aligned}$$

$$RTO = RTT_s + 4RTT_D = 1.625s + 4 \times 0.78125s = 4.75s$$

5-37

(1) 慢开始：一种 TCP 拥塞控制算法。其在主机刚刚开始发送报文段时可先将拥塞窗口 **cwnd** 设置为一个最大报文段 **MSS** 的数值。在每收到一个对新的报文段的确认后，将拥塞窗口增加至多一个 **MSS** 的数值。

作用：用这样的方法逐步增大发送方的拥塞窗口 **cwnd**，可以使分组注入到网络的速率更加合理。

(2) 拥塞避免：一种 TCP 拥塞控制算法。让拥塞窗口 **cwnd** 缓慢地增大，即每经过一个往返时间 **RTT** 就把发送方的拥塞窗口 **cwnd** 加 1，而不是加倍，使拥塞窗口 **cwnd** 按线性规律缓慢增长。因此在拥塞避免阶段就有“加法增大”的特点。在拥塞避免阶段，拥塞窗口 **cwnd** 按线性规律缓慢增长，比慢开始算法的拥塞窗口增长速率缓慢得多。

作用：可以迅速减少主机发送到网络中的分组数，使得发生拥塞的路由器有足够时间把队列中积压的分组处理完毕。

(3) 快重传：一种 TCP 拥塞控制算法。快重传算法首先要求接收方不要等待自己发送数据时才进行捎带确认，而是要立即发送确认，即使收到了失序的报文段也要立即发出对已收到的报文段的重复确认。发送方只要一连收到三个重复确认，就知道接收方确实没有收到报文段，因而应当立即进行重传（即“快重传”），这样就不会出现超时，发送方也不就会误认为出现了网络拥塞。

作用：使用快重传可以使整个网络的吞吐量提高约 20%。

(4) 快恢复：一种 TCP 拥塞控制算法。当发送端收到连续三个重复的确认时，由于发送方现在认为网络很可能没有发生拥塞，因此现在不执行慢开始算法，而是执行快恢复算法 FR (Fast Recovery) 算法：(a) 慢开始门限 $ssthresh = \text{当前拥塞窗口} / 2$ ；(b) 新拥塞窗口 $cwnd = \text{慢开始门限 } ssthresh$ ；(c) 开始执行拥塞避免算法，使拥塞窗口缓慢地线性增大。

作用：使拥塞窗口缓慢地线性增大。

(5) 加法增大：在拥塞避免阶段，拥塞窗口是按照线性规律增大的。

(6) 乘法减小：当出现超时或 3 个重复的确认时，就要把门限值设置为当前拥塞窗口值的一半，并大大减小拥塞窗口的数值。